

Storage Wars

Your Postgres driver has been padding with old secrets, and Cloudflare builds a log out of object storage

Issue #6

Sat, 10 Oct 2026

🕒 ~37 min read

Level 3/5

📰 Today's two stories

1. **The PostgreSQL JDBC driver had two security bugs, both fixed in 42.7.14 (7 Oct).** In one, if your code promised the driver more bytes than it then supplied, the driver made up the difference with leftover bytes from earlier traffic on the connection, such as SQL text and parameter values, and the server saved them. That affects every version from 42.7.4 (August 2024) to 42.7.13. In the other, a `requireAuth` value that rules out every login method ended up enforcing nothing at all. We cover how the Postgres wire protocol forces a driver to pad, where the stale bytes come from, the run of authentication bugs before this one, and what to check in your own code.
2. **Cloudflare launched K2, a serverless event stream built directly on R2 object storage (public beta, 1 Oct).** It has no brokers or disks to manage and is priced per GB, but it adds about a second of produce latency. We explain why object storage can't do appends, how K2 batches writes into segment files, how its lease-based consumers differ from Kafka consumer groups, and where it sits in the wider move toward "diskless" streaming, including Kafka's own KIP-1150.
3. **On the radar:** 7 more things that happened, listed at the end.

STORY 1 The JDBC driver that padded your data with someone else's

PGJDBC 42.7.14 · RELEASED 7 OCT 2026 · CVE-2026-107315 AND CVE-2026-107314 · ~17 MIN READ

Imagine you tell a courier the parcel is a metre long, then hand over something half that size. A careful courier pads the box with clean packing paper. For about two years the PostgreSQL JDBC driver padded it with scraps from the last few parcels it had carried instead: earlier SQL statements, earlier parameter values, possibly from a different request that had borrowed the same pooled connection. And the database stored those scraps as part of your data.

WHAT HAPPENED

On **7 October 2026** the PostgreSQL JDBC project, usually called **pgjdbc**, shipped version **42.7.14** as a security release. The postgresql.org announcement, posted on 9 October, lists two CVEs and their GitHub Security Advisories: **CVE-2026-107314** (GHSA-rhp9-mr79-r74h) and **CVE-2026-107315** (GHSA-f64h-wr5q-3qf3).¹ The project's homepage now lists 42.7.14, dated 7 October, as the current release.²

The project's security page summarises both bugs:³

- **CVE-2026-107315, "padding with earlier traffic"**. When a value is shorter than its declared length, the driver fills the gap with bytes from earlier messages on the connection instead of with zeros, so the server can end up storing SQL text or parameter values. **Affected: 42.7.4 through 42.7.13**. Versions 42.7.3 and earlier pad with zeros.
- **CVE-2026-107314, "requireAuth not enforced"**. If the `requireAuth` connection property excludes all six authentication methods the driver knows, the driver applies no restriction at all and will accept a cleartext password request from the server. **Affected: 42.7.11 through 42.7.13**.

Both are fixed in 42.7.14. The CVE records rate them Medium: **5.3** for the padding bug and **5.9** for the `requireAuth` bug, on the CVSS 3.1 scale.^{5,6} The CVE records credit Vladimir Sitnikov as the finder of the padding bug and Daniel Coles as the reporter of the `requireAuth` bug.^{5,6}

The same update to the security page also documented an older, already-fixed bug of the same family: **CVE-2026-107313**, in which, on GSS-encrypted connections, 42.7.4 and 42.7.5 could send old send-buffer contents in place of the first 16,320 bytes of a large value. That one was fixed back in **42.7.6**.^{3,4}

Downstream projects started moving quickly. Camunda, for example, merged a bump to 42.7.14 on its `stable/8.6` branch on 9 October. That branch had been on 42.7.11, inside the `requireAuth` window, although the commit message argues the vulnerable path is unreachable because that release line never opens a PostgreSQL connection.¹⁰

Neither bug is a remote "take over the database" hole. But one quietly writes earlier traffic into your tables, and the other turns a security setting into a no-op. Both are worth understanding, because both come from the small print of how a driver talks to Postgres.

THE BACKGROUND YOU NEED

What a JDBC driver actually does

JDBC *≈ Java Database Connectivity, the standard Java API (`Connection` , `PreparedStatement` , `ResultSet`) that every Java database driver implements* is just a set of interfaces. The code that actually talks to PostgreSQL is the driver, and for Postgres that's `pgjdbc`, published on Maven Central as `org.postgresql:postgresql`.¹ If your Java, Kotlin or Scala service uses Postgres through Hibernate, Spring Data, jOOQ, MyBatis or plain JDBC, this jar is almost certainly underneath.

The driver turns your method calls into messages in the **wire protocol** *≈ the binary message format that PostgreSQL clients and servers exchange over the network connection* . You rarely think about that layer, but this story lives there.

The protocol makes you say how long a value is before you send it

When you run a prepared statement, the driver sends a **Bind** message that fills in the parameter values. Here's the relevant part of Bind's layout, from the PostgreSQL documentation:¹¹

- `Int32` : "The length of the parameter value, in bytes (this count does not include itself)." A value of -1 means NULL.
- `Byte n`: "The value of the parameter ... *n* is the above length."

COPY works the same way: each **CopyData** message carries an `Int32` length "of message contents in bytes, including self", followed by the data.¹¹

So the length goes on the wire **first**, and the bytes follow. That ordering is what makes **message framing** *≈ how the receiver knows where one message ends and the next begins; here, by trusting the declared length* work. The server reads the length, then reads exactly that many bytes and treats them as the value. It has no other way to know where the value stops.

Now suppose the driver has already sent "the next value is 1,000 bytes long" and your code only produces 600. The driver can't take the length back: it's already in the buffer, maybe already on the network. If it just stopped, the server would read the next 400 bytes of the following message as part of your value, and the connection would fall apart. So the driver has to send 400 more bytes of *something*. That's padding. The only question is what goes in it.

When can a value be shorter than declared?

In most JDBC calls, the driver knows the exact length because you handed it a `String` or a `byte[]`. The risky APIs are the ones where you declare a length and then stream the data yourself. The advisory names five of them:^{5,8}

- `PreparedStatement.setObject` with a `ByteStreamWriter`
- `CopyIn.writeToCopy`
- `PGCopyOutputStream.write`
- `LargeObject.write`
- `Blob.setBytes`

ByteStreamWriter *≈ a pgjdbc interface for sending a binary parameter by writing to an OutputStream, so you don't have to build one big byte array on the heap* is the clearest example. Its Javadoc says you implement two methods: `getLength()`, which returns the number of bytes, and "The length must be known ahead of the stream being written to"; and `writeTo(target)`, which writes the data and "should not write more than `getLength()` bytes".¹² The interface stops you writing *more* than you promised. Before 42.7.14, nothing stopped you writing *less*.

Large objects *≈ PostgreSQL's older facility for storing big binary values outside normal rows, written and read in chunks through a special API* and **COPY** *≈ PostgreSQL's bulk-load command, which streams many rows to the server in a single operation* have the same shape. You pass an `(array, offset, length)` triple, and if the length runs past the real data, the driver has to fill in the rest.

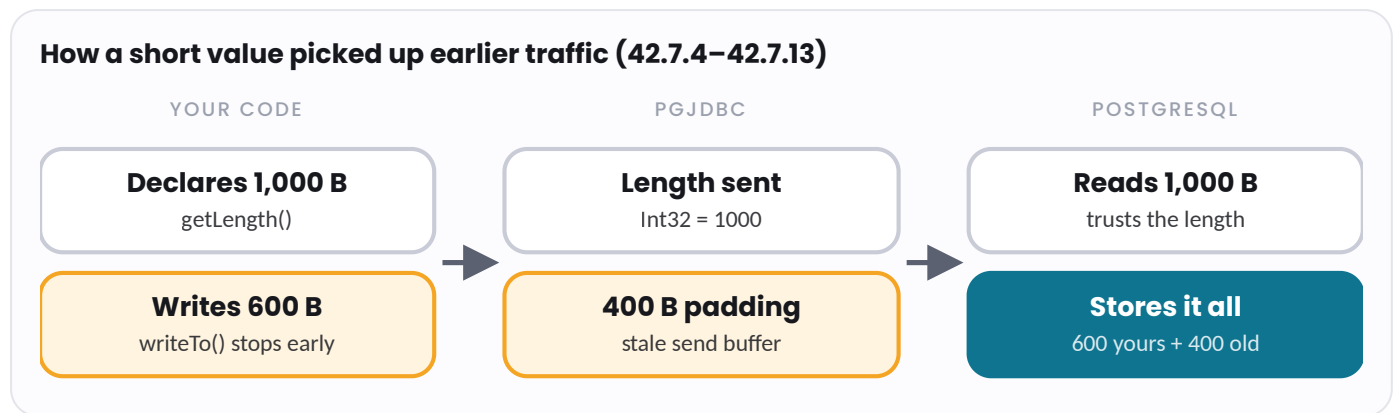
HOW IT WORKS

Where the stale bytes came from

Here's the key sentence from the advisory: the driver "pads a value that is shorter than its declared length with bytes left in its send buffer instead of zeros".⁸

A **send buffer** *≈ a block of memory where the driver assembles outgoing messages before writing them to the network socket in one go* is reused for every message on a connection. After a few statements it holds the remains of earlier Bind messages, query strings and parameter values. If padding copies "whatever is in the buffer right now" instead of writing fresh zeros, those remains are what gets sent.

The CVE record spells out the consequences. The leftover bytes "can include earlier SQL text and parameter values", "possibly from other requests on a pooled connection". Each padded value "can carry up to 8192 bytes" of that traffic, "or 16320 bytes with GSS encryption".⁵



The numbers above are illustrative. The mechanism and the per-value limits come from the advisory.⁵

Why did this start in 42.7.4? The advisory doesn't name the commit that introduced it. But the 42.7.4 changelog is suggestive. That release (announced 23 August 2024¹⁴) replaced Java's `BufferedOutputStream` with a new unsynchronized `PgBufferedOutputStream`, removed a separate `PGStream.streamBuffer` so that `PgBufferedOutputStream`'s own buffer is reused when sending data from an `InputStream`, added a `maxSendBufferSize` connection property, and "improved `writeZeros` tests".¹³ A rewrite of the write path, a reused buffer and new tests for the zero-writing code all landing in the first affected release looks like the likely origin. That is our reading of the changelog, not something the advisory says.

The GSS variant fits the same pattern. CVE-2026-107313 describes old contents of the "GSS send buffer" being sent in place of a large value on encrypted connections in 42.7.4 and 42.7.5.³ **GSS encryption** *≈ encrypting the Postgres connection using Kerberos (via the GSSAPI) instead of TLS* has its own send buffer, and its leak limit (16,320 bytes) is larger than the normal one (8,192).

Who can actually exploit it?

This isn't a bug an attacker can trigger from outside on its own. Three things have to be true at once, according to the CVE record:⁵

1. Your application uses one of the five APIs and **declares a length larger than the data it supplies**. The driver accepted that "without error".
2. An attacker can get your application to **store** such a value.
3. The attacker can **read that value back**: download the file, view the blob, export the table.

Applications "whose declared lengths always match their data are not affected".⁵ So the realistic victim is something like a document service that streams uploads into a `bytea` column or a large object, with a length

taken from an HTTP `Content-Length` header or a metadata field that can disagree with the real body. If an attacker can make the declared length bigger than the data and then download their own "file", they get up to 8 KB of whatever the connection carried before, per upload. On a pooled connection that could be another user's query parameters. The CVSS vector reflects this: network-reachable, **high** attack complexity, low privileges, **high** confidentiality impact, no integrity or availability impact.⁵

There's a second, quieter consequence: even with no attacker at all, a buggy length calculation means your database has been storing values with junk on the end. That junk can include secrets, and it ends up in backups and replicas.

The other bug: a filter that filtered nothing

`requireAuth` lets the client decide which authentication methods it will accept from the server. That matters because, in Postgres, the **server** picks the method. A fake server sitting between you and the real one (a **man-in-the-middle** *≈ an attacker who intercepts the connection and pretends to be the server to the client, and the client to the server*) can simply ask for a cleartext password. An **AuthenticationCleartextPassword** *≈ the server message (R with code 3) that tells the client to send its password unencrypted* request is just a message with code 3.¹¹ A client that answers it hands over the password.

The idea came from libpq, the C client library. PostgreSQL 16 added a `require_auth` parameter, in a commit authored by Jacob Champion and dated 14 March 2023. It takes a comma-separated list of `password`, `md5`, `gss`, `sspi`, `scram-sha-256` and `none`. A `!` prefix turns an entry into an exclusion, you can't mix the two forms in one list, and if the server asks for a method that isn't allowed, the connection fails before any secret is sent.¹⁵ `pgjdbc`'s `requireAuth` follows the same idea.

The bug is a classic **fail-open** *≈ a security check that, in an unexpected case, lets everything through instead of blocking everything* edge case. The CVE record explains: in 42.7.11–42.7.13, if the value excludes all six known methods, for example `!password,!md5,!gss,!sspi,!scram-sha-256,!none`, or contains no method at all (a lone comma), "the driver then accepts whatever method the server requests, including cleartext password authentication".⁶ The weakness is filed as CWE-636, "Not Failing Securely ('Failing Open')".⁷ Positive lists like `requireAuth=scram-sha-256` and partial exclusions like `!password,!md5` were enforced correctly, and because the property has no default, deployments that never set it aren't affected.⁶

requireAuth values in 42.7.11–42.7.13

Fail open (no check at all)

```
# excludes every method:
requireAuth=!password,!md5,!gss,
!sspi,!scram-sha-256,!none
# names no method:
requireAuth=,
# 42.7.14: refused (08004)
```

Enforced correctly

```
# allow-list (recommended):
requireAuth=scram-sha-256
# partial exclusion:
requireAuth=!password,!md5
# pair with TLS checks:
sslmode=verify-full
```

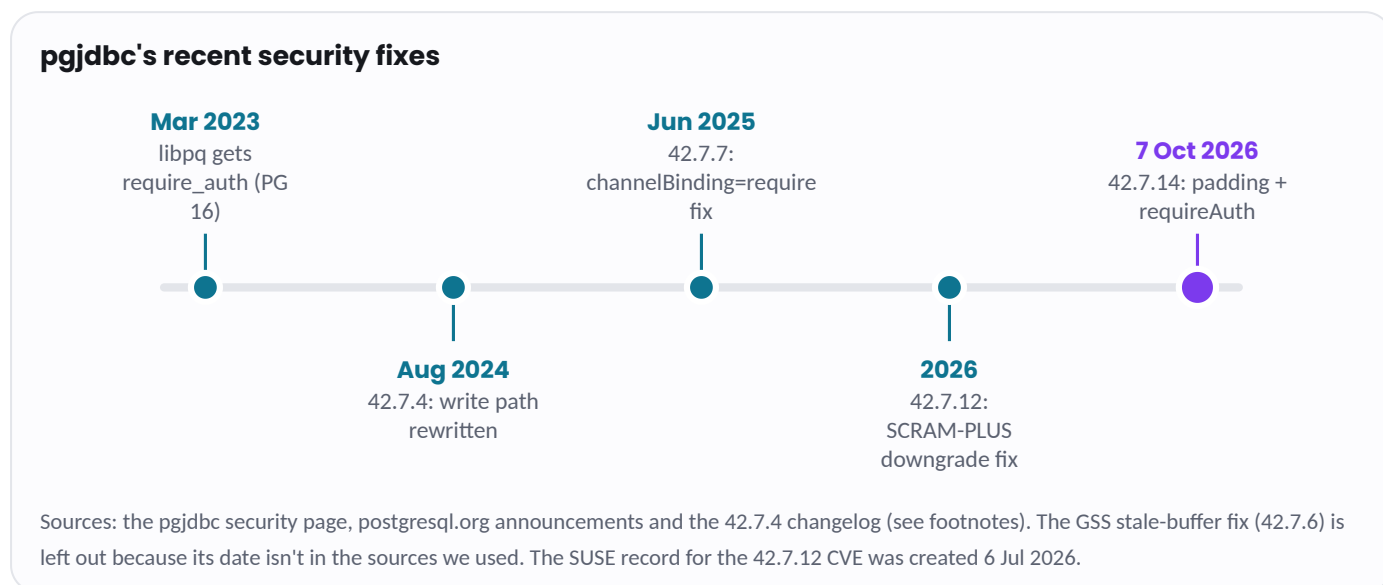
Behaviour from the CVE record for CVE-2026-107314. Values are shown wrapped for width.

Who would write a list that excludes everything? Probably nobody on purpose. It's what you get from config templating gone wrong, an empty environment variable, or an "exclude everything we don't trust" list built by a script. The point of a security setting is that its strange cases fail safe. In 42.7.14 they do: such connections are refused with `SQLState` `≈ the standard five-character error code JDBC drivers attach to SQL exceptions` `08004`, and an empty method value is rejected as invalid.⁷

A pattern: "the setting didn't do what it said"

This is the third time in about 16 months that a pgjdbc connection setting meant to resist a man-in-the-middle turned out to be weaker than its name.

Channel binding `≈ a SCRAM authentication feature that ties the login to the specific TLS connection, so a man-in-the-middle with a different certificate can't relay it` is the other protection in this area. In June 2025, CVE-2025-49146 found that with `channelBinding=require`, pgjdbc "would incorrectly allow connections to proceed with authentication methods that do not support channel binding (such as password, MD5, GSS, or SSPI authentication)". It was fixed in 42.7.7.¹⁶ Then CVE-2026-54291 found that 42.7.4 through 42.7.11 could silently downgrade from **SCRAM-SHA-256-PLUS** `≈ the SCRAM password login with channel binding switched on` to plain SCRAM-SHA-256 when the server certificate used a signature algorithm with no channel-binding hash, such as Ed25519. That was fixed in 42.7.12.^{3,17}



None of these is a disaster on its own. Together they make a useful lesson: a client-side security setting is only as good as its tests for the odd cases. "Exclude everything", "server picks an unusual certificate" and "the stream ended early" are exactly the cases nobody writes a test for until someone files a CVE.

WHAT IT MEANS FOR YOU

Upgrade to 42.7.14. That's the fix for both bugs, and it's a patch release in the same 42.7.x line you're probably already on. In Maven or Gradle that means bumping `org.postgresql:postgresql` to `42.7.14`.¹

If you use Spring Boot, the driver's version comes from Boot's dependency management. You can override it with the `postgresql.version` property (for example `<postgresql.version>42.7.14</postgresql.version>` in

a Maven POM) until your Boot version picks it up.¹⁸ Check what you actually run with `mvn dependency:tree` or `gradle dependencies`. A transitive dependency can easily pin an older driver.

If you can't upgrade today, the advisories describe what makes you safe:

- For the padding bug, make every declared length match the data. `ByteStreamWriter.getLength()` must equal the number of bytes `writeTo()` writes, and the `(offset, length)` pairs you pass to the byte-array methods must stay inside the array. There's no connection property that turns the behaviour off.⁸ Search your code for `ByteStreamWriter`, `CopyIn`, `PGCopyOutputStream`, `LargeObject` and `Blob.setBytes`. If none of them appear, you're very likely not exposed.
- For `requireAuth`, grep your connection strings and config for the property. An allow-list such as `requireAuth=scram-sha-256` is fine. A value that excludes every method, or is empty, is the problem. Combining it with `sslmode=verify-full` also means a man-in-the-middle has to beat certificate verification before it can ask for a password at all.

If you think you were exposed to the padding bug, look at stored values written through those APIs since you moved to 42.7.4 or later. If any of them were shorter than declared, their tails may contain earlier traffic. That data is in your backups and replicas too. Fixing the driver stops new leaks but doesn't clean old rows.

In the real world

Camunda's bump on its 8.6 branch is a good template for triage. The commit message names the CVE, notes that the branch was on 42.7.11 (inside the affected range), explains why the team believes the vulnerable path is unreachable (DataSource auto-configuration is excluded on that line, so it never opens a PostgreSQL connection), and upgrades anyway. The bump was manual because Renovate doesn't cover that branch.¹⁰ Most teams will be in the same position: probably not exploitable, cheap to fix, so fix it.

Watch out for

- **"Newer is safer" didn't hold here.** 42.7.3 and earlier padded with zeros. The padding leak was introduced in 42.7.4. Downgrading isn't the fix. 42.7.14 is.
- **Scanners may show odd version ranges.** One aggregator lists only 42.7.4–42.7.8 as affected by the padding bug, while the CVE text says 42.7.4–42.7.13.⁹ Go by the CVE text.
- **Severity labels differ.** The CVSS 3.1 scores are Medium (5.3 and 5.9),^{5,6} but some feeds label the padding advisory "High".⁹ Your scanner's triage queue may sort them differently.
- **NVD analysis was still pending** when we checked, so expect scores and CPE data to change.⁵

WHAT HAPPENS NEXT

The pgjdbc changelog on GitHub already has an "Unreleased" section with more fixes, mostly around XA transactions and `PGStream.skip`, so another 42.7.x will follow.¹⁹ The bigger open question is testing. Most of the recent pgjdbc security fixes came from edge cases in features that work fine on the normal path. Expect the project, and other drivers that copied libpq's `require_auth` idea, to look harder at their "everything excluded" and "short stream" cases.

Words from this story

JDBC: Java Database Connectivity, the standard Java API that every Java database driver implements.

Wire protocol: the binary message format PostgreSQL clients and servers exchange over the connection.

Message framing: how the receiver knows where one message ends and the next begins; here, by trusting the declared length.

ByteStreamWriter: a pgjdbc interface for sending a binary parameter by writing it to an OutputStream.

Large objects: PostgreSQL's older facility for storing big binary values outside normal rows, written in chunks.

COPY: PostgreSQL's bulk-load command, which streams many rows in one operation.

Send buffer: memory where the driver assembles outgoing messages before writing them to the socket.

GSS encryption: encrypting the Postgres connection with Kerberos (GSSAPI) instead of TLS.

Man-in-the-middle: an attacker who intercepts a connection and impersonates each side to the other.

AuthenticationCleartextPassword: the server message asking the client to send its password unencrypted.

Fail-open: a security check that lets everything through in an unexpected case instead of blocking.

SQLState: the standard five-character error code attached to SQL exceptions.

Channel binding: a SCRAM feature that ties the login to the specific TLS connection.

SCRAM-SHA-256-PLUS: SCRAM password authentication with channel binding switched on.

STORY 2 Cloudflare K2: a Kafka-style log with no brokers and no disks

CLOUDFLARE K2 · PUBLIC BETA, ANNOUNCED 1 OCT 2026 · EVENT STREAMING ON R2 · ~15 MIN READ

For a decade, "event streaming" has meant Kafka, and Kafka has meant brokers: long-running servers with fast local disks, copying every byte to each other across data centres. Cloudflare just launched a streaming service with none of that. No brokers to size, no disks to fill, no partitions to rebalance. Its log lives entirely in object storage, the same kind of storage you use for images and backups. The price is about a second of latency on every write. Whether that's a bargain depends on what you stream.

WHAT HAPPENED

On **1 October 2026**, Cloudflare announced **K2** in public beta. The post, by Micah Wylde and Marc Selwan, describes K2 as a "fully serverless" durable event-streaming service that "implements a partitioned, durable log on top of R2 object storage".²⁰ InfoQ covered it on 9 October, along with the Hacker News discussion that followed.²¹

The basics, from the launch post and the docs:

- **Model:** producers write records to a **stream**, which keeps them as an ordered log for a retention period you choose. Consumers read through **subscriptions**, either sharing the work or each getting every record.^{20,22}
- **Interfaces:** produce through a Workers binding or an HTTP API. Consume over HTTP with a lease-based API. Create streams from the dashboard, the API, Wrangler, or Cloudflare's new `cf CLI`.²⁰
- **Latency:** the first release adds "about 1 second of produce latency at the 99th percentile".²⁰ That's Cloudflare's own figure.
- **Availability:** Workers Paid plan only.²⁰
- **Price after the beta:** \$0.04 per GB produced, \$0.04 per GB consumed and \$0.02 per GB-month retained. Nothing is billed during the beta.²⁰

The limits page lists 20 streams per account, retention between 1 hour and 30 days (default 7 days), 30 MB/s of produce throughput per stream, records of up to about 1 MB and 5 MB per produce request.²³ There's one inconsistency to note: the launch post says the beta allows "10GB of storage" **per stream**,²⁰ while the limits page says 10 GB **total across all streams** in an account.²³ Plan for the stricter one.

THE BACKGROUND YOU NEED

What an event stream is for

An event stream *≈ an append-only, ordered log of records that many independent readers can consume at their own pace* sits between the systems that produce events and the systems that use them. Cloudflare's example is an online shop: every transaction event goes to an analytics system and a fraud-detection service. Without a buffer in between, the post points out, "events are dropped" whenever producers outrun consumers or a downstream

service goes offline.²⁰ The stream absorbs the writes and lets each reader catch up in its own time. That decoupling is why most companies above a certain size run Kafka or something like it.

How Kafka does it, and what that costs

In **Apache Kafka** *≈ the open-source distributed event-streaming platform; the de facto standard for event logs*, a topic is split into **partitions** *≈ independent ordered logs that together make up a topic; each is owned by one broker at a time*. Each partition lives on the local disk of a **broker** *≈ a Kafka server that stores partitions and serves reads and writes for them*, and is copied to other brokers, usually in other **availability zones** *≈ separate data centres within one cloud region, with independent power and networking*, so that losing one machine loses nothing.

That design is fast, since you're writing to a local disk, and battle-tested. But in the cloud, the replication has a price tag. Kafka's own proposal for "diskless" topics, **KIP-1150**, says replicating active segments across zones is a major infrastructure cost for operators on the big clouds. It quotes cross-zone traffic at **\$0.02 per GiB on AWS** and **\$0.01 per GiB on Google Cloud**, while Azure doesn't charge for it.²⁷ Every byte you produce gets copied to other zones, and on AWS every copy costs money. On top of that are the operational costs: rebalancing data when you add brokers, sizing disks, replacing failed nodes.

The object-storage alternative

Object storage *≈ storage you use through HTTP PUT and GET of whole objects (S3, R2, GCS); very durable and cheap, but slower per request than a local disk* services already replicate data across zones and charge you for storage, not for internal copying. Cloudflare's R2 is "designed to provide 99.99999999% (eleven 9s) of annual durability", stores data across several data centres in a region, and only returns **HTTP 200** for a write "once the data has been persisted to disk".²⁵ So if a streaming system can keep its log in object storage, durability and replication become someone else's problem, and brokers become stateless processes you can add or remove freely.

The idea isn't new. **WarpStream** built a Kafka-compatible platform on object storage and was bought by Confluent on 9 September 2024. Confluent positioned it for "high-scale workloads with relaxed latency requirements", such as logging, observability and feeding data lakes.²⁹ Kafka itself followed. KIP-1150 was accepted on **2 March 2026**, with 9 binding votes, as the umbrella for a new topic type that writes data through to object storage and uses local disks mainly as a cache. The detailed design lives in follow-up KIPs, KIP-1163 (Diskless Core) and KIP-1164 (Diskless Coordinator), which were still being worked on when Aiven wrote about the vote.^{28,27} KIP-1150 is explicit about the cost: diskless topics "will incur higher produce latency" than classic ones.²⁷

Event streams move onto object storage



Sources: Confluent press release, Kafka 4.2.0 announcement, Aiven and the KIP-1150 page, Cloudflare's K2 post.

Why Cloudflare couldn't just run Kafka

K2 didn't start as a product. Cloudflare built it first as the durable buffer behind **Pipelines**, its serverless ingestion service (part of the Basin data platform we covered in issue #2). Pipelines runs on the Arroyo stream-processing engine, which *pulls* data, so accepted events have to be stored durably until Arroyo reads them. And Pipelines promises never to drop an accepted event.²⁰

Kafka was a poor fit for where that buffer had to run. Cloudflare's network spans "over 335 cities", and its stateful services get small, short-lived slices of machines that talk to each other over the public internet, conditions in which, the post says, traditional distributed systems like Kafka are often impractical.²⁰ Building on R2 meant not running that kind of software at all.

HOW IT WORKS

Problem one: object stores can't append

A log is an append-only data structure, and object stores don't support appends. You can write a whole new object, but you can't add ten bytes to the end of an existing one. Writing one object per event would be ruinous anyway, because object stores charge per request. R2's list price is **\$4.50 per million** "Class A" operations, which include `PutObject`.²⁶

K2's answer is to batch. Writes collect in memory on an edge service, and "after a short wait for data to arrive", all of them are written to R2 as a single segment file *≈ one object in storage holding a contiguous run of records from the log*, large enough to spread the per-object cost.²⁰ That short wait is where the second of produce latency comes from. It's a dial between latency and cost, and Cloudflare has set it to "about a second" for now.

How much does that dial matter? Here's a back-of-the-envelope calculation using R2's public PUT price. It's our arithmetic, not Cloudflare's: K2's internal costs and exact flush interval aren't published.

Cost of PUTs for one stream at R2 list price, per 30 days



Illustrative, derived from R2's \$4.50 per million Class A operations (<https://developers.cloudflare.com/r2/pricing>): $\text{PUTs/s} \times 86,400 \times 30 \times \$4.50 / 1,000,000$. Not K2's actual costs.

Flushing every 10 ms instead of every second multiplies the request bill by 100, for every stream. That's the basic economics of object-storage streaming. A competitor said as much in the Hacker News thread: "One of the tensions of course is how long to linger before flushing to object storage."²¹

Problem two: ordering without a coordinator

A log also needs ordering: every record gets a strictly increasing **offset** *≈ a record's position in the log; consumers track how far they've read by offset*. In Kafka, the partition's leader broker hands those out. Many object-storage designs add a separate metadata service or database to do it.

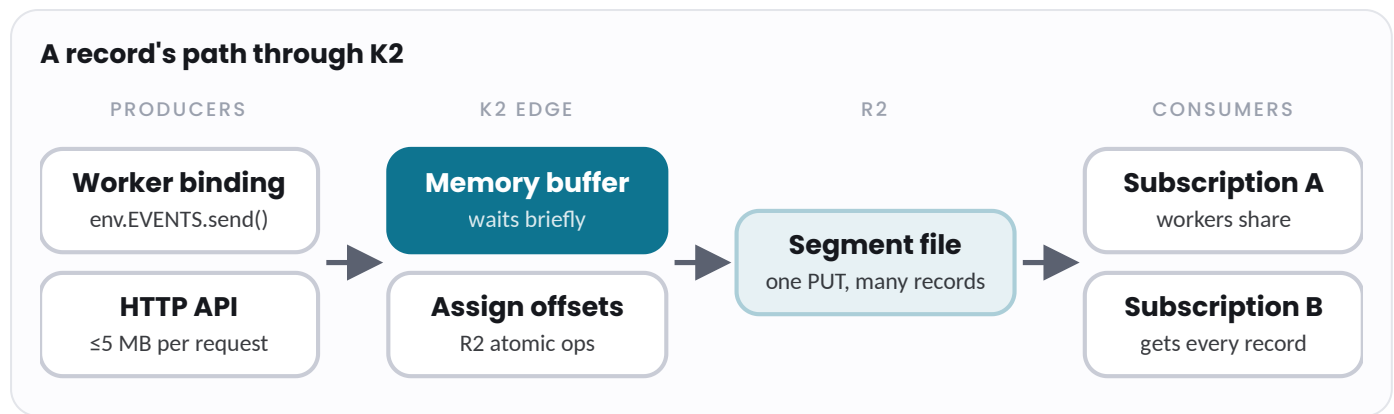
K2 skips that. The post says it uses R2's "atomic operations" to get ordering and strictly increasing offsets, so it doesn't need a separate coordination service.²⁰ It doesn't say which operations. Object stores commonly offer conditional writes, where a write only succeeds if the object doesn't exist yet or hasn't changed, and that's the usual building block for this kind of trick. But Cloudflare hasn't said that's what K2 uses and has promised a separate technical deep dive.²⁰

Producing

The launch post shows a Worker producing through a binding.²⁰ Adapted from that example, it looks like this:

```
const result = await env.EVENTS.send([
  { content: JSON.stringify({ type: "page_view", path: url.pathname }) },
]);
if (!result.success) {
  console.error(result.error);
  return new Response("try later", {
    status: result.error.retryable ? 503 : 500,
  });
}
```

K2 stores record content as plain bytes, so choosing JSON, Avro or Protobuf is up to you.²⁰ Records can also carry up to 32 headers.²³



Consuming: leases, not partitions

This is where K2 differs most from Kafka. A Kafka consumer group assigns whole partitions to consumers and each consumer commits offsets. K2's consumer API doesn't expose partitions at all. Instead it works more like a job queue:²⁴

1. You create a **subscription** with `start_at` set to `earliest` (oldest retained record) or `latest`. Subscriptions can't be changed after creation, and a stream can have up to 100.
2. A worker calls `consume` with a `worker_id` and `max_records` (up to 10,000). It gets back a `batch_id`, the records, and `leased_until_ms`.
3. The batch is leased to that worker for **five minutes**. K2 "never leases the same records to two workers at once", and a subscription can have up to 128 active leases.
4. The worker then **acks** the batch (done), **nacks** it (send it to someone else now), or **extends** the lease by another five minutes. If the lease expires, the records go to the next worker that asks.

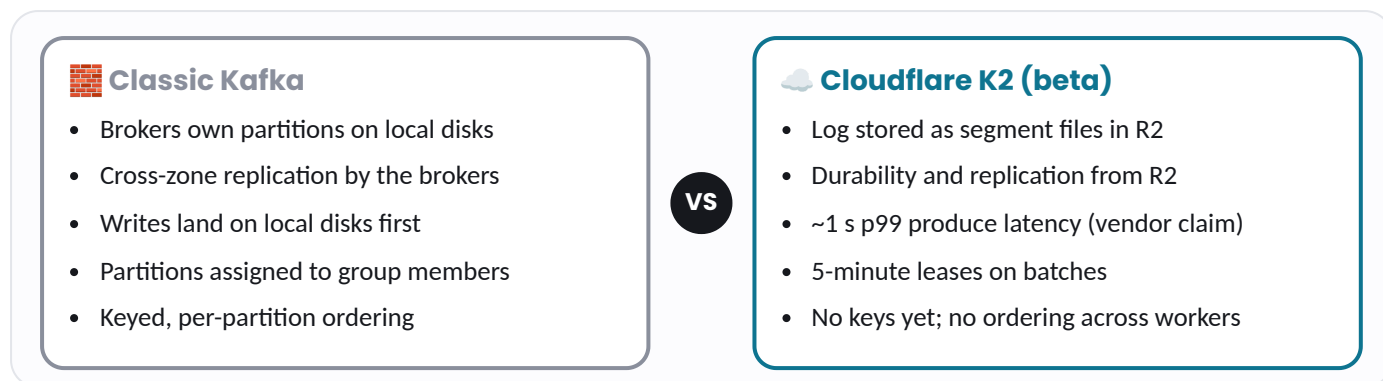
```
curl "$K2_ENDPOINT/subscriptions/$SUB_ID/consume" \
  --request POST \
  --header "Authorization: Bearer $CLOUDFLARE_API_TOKEN" \
  --header "Content-Type: application/json" \
  --data '{"worker_id": "worker-1", "max_records": 100}'
```

Two details are easy to miss. First, a nacked batch blocks progress: "The subscription does not advance past these records until the redelivered batch is acked."²⁴ One poison batch that always fails can stall a whole subscription, and there's no dead-letter queue. Second, the guarantees are modest. Delivery is **at least once**, so your consumer must cope with duplicates, and ordering is not guaranteed across workers sharing a subscription.²⁴ **At-least-once delivery** *≈ every record is delivered one or more times, so duplicates are possible and consumers must be idempotent* is normal for streaming, but losing ordering across workers is a real difference from Kafka, where each partition is read in order by exactly one consumer in a group.

Why this design? The K2 tech lead, posting on Hacker News, said he finds the Kafka API "too low-level for typical users and too high-level for deep integration with stream processing engines". The team argues that leases suit Workers, which "parallelize well but are individually weak": many small workers can each grab a batch, without partitions limiting how many can read at once.²¹ Kafka has been moving in a similar direction. Its **share groups** ("Queues for Kafka"), declared production-ready in Kafka 4.2 in February 2026, bring queue-

style consumption with acquisition locks on records, and KIP-1222 added a way to "extend acquisition lock timeouts on records requiring longer processing times".³⁰

Lease *≈ a time-limited claim on some work; if the holder doesn't finish or renew it in time, the work is handed to someone else* is the key word in both designs: neither trusts a consumer to stay alive, so unfinished work goes back to the pool when a lease or lock runs out.



What the first testers saw

The Hacker News thread produced the first independent numbers, and they're less flattering than the headline. One beta user reported writes under a second, but **end-to-end** delivery (produce to consume) of **2.5 seconds at p95 and 7.5 seconds at p99**. Someone answering for the team said the tail was higher than they'd expect, that the consume API needs performance work, and that read-latency improvements should land within about a week.²¹ That's one user's measurement with an undescribed setup, so treat it as an early data point, not a benchmark.

A commenter who disclosed that he works on competitor S2 (s2.dev) argued that object-storage streams can do far better: faster single-zone tiers like S3 Express still need quorum writes across zones for regional durability, but he said his service reaches about 50 ms p99 acknowledgement in the same region.²¹ That's a competitor's own claim, but it shows how much the batching window, not object storage itself, decides the latency.

WHAT IT MEANS FOR YOU

K2 is a good fit when latency in seconds is fine and you're already on Cloudflare. Think analytics events, logs, clickstreams, audit trails, fan-out to a few slow consumers, or a buffer in front of a data lake. If the data ends up as files or Iceberg tables in R2 anyway, Cloudflare itself recommends Pipelines rather than K2. K2 is for custom processing or other destinations.²⁰ For individual work items that need retries, delays and dead-letter queues, Cloudflare points you to its Queues product instead.²⁰

It's not a Kafka replacement yet. There are no Kafka clients (they're on the roadmap), no message keys or key-based ordering, retention tops out at 30 days unless you ask for more, and the beta caps storage at 10 GB.^{20,23} If your consumers depend on per-key ordering, such as applying updates to an account in sequence, K2 can't give you that today.

Do the cost maths with fan-out in mind. At \$0.04 per GB produced plus \$0.04 per GB consumed, one consumer costs \$0.08 per GB moved, and each extra subscription adds another \$0.04.²⁰ A commenter pointed

out exactly this, though the same commenter also argued K2 would still undercut Google Pub/Sub for long retention and managed Kafka for the effort saved.²¹ Five subscriptions on 1 TB a day is about \$240 a day at list price (\$40 produce plus \$200 consume), before retention. That's our arithmetic from the published prices.

Watch out for

- **Beta limits disagree.** The launch post says 10 GB storage per stream; the limits page says 10 GB per account. Assume the smaller.^{20,23}
- **One stuck batch stalls a subscription.** Naked records block progress until they're acked, and there's no dead-letter queue. Build your own "give up and park it" logic.²⁴
- **Don't retry 10218.** If extending a lease fails with that code, the lease is gone. Stop processing and fetch a new batch, or you'll process records another worker now owns.²⁴
- **Latency numbers are early.** Cloudflare's ~1 s p99 is for produce, not end-to-end, and is a vendor figure. One tester saw 7.5 s p99 end to end.²¹

WHAT HAPPENS NEXT

Cloudflare's roadmap lists multi-GB/s write parallelism, message keys with key-based ordering, push-based Worker consumers, a lower-latency **Express** tier, and drop-in support for Apache Kafka clients.²⁰ The Express tier is the one to watch: it's an admission that a second is too slow for many uses, and S2's numbers suggest how far object-storage designs can go. Meanwhile, Kafka's own diskless topics still need KIP-1163 and KIP-1164 to land before you'll see them in a release.²⁸ If they do, you'll get roughly the same trade-off (cheaper, slower writes) on a per-topic basis inside Kafka.

Words from this story

Event stream: an append-only, ordered log of records that many readers consume at their own pace.

Apache Kafka: the open-source distributed event-streaming platform; the de facto standard for event logs.

Partitions: independent ordered logs that together make up a Kafka topic.

Broker: a Kafka server that stores partitions and serves reads and writes for them.

Availability zones: separate data centres in one cloud region with independent power and networking.

Object storage: storage used via HTTP PUT/GET of whole objects (S3, R2, GCS); durable and cheap but slower per request.

Segment file: one stored object holding a contiguous run of log records.

Offset: a record's position in the log, used by consumers to track progress.

At-least-once delivery: every record is delivered one or more times, so consumers must handle duplicates.

Lease: a time-limited claim on work that goes back to the pool if not finished or renewed.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **PostgreSQL 19 RC still pending:** as of 10 October, the newest PostgreSQL 19 release announcement on postgresql.org is still Beta 4 (24 Sep).³¹
- **Kafka 4.4.0 still unannounced:** the newest post on the Apache Kafka blog is still the 4.2.2 bugfix release (29 Sep).³²
- **MongoDB 9.0 known issue:** `listIndexes` now reports `collation: { locale: "simple" }` on simple-collation indexes, which can make ODMs such as Mongoose drop and rebuild indexes on `syncIndexes()`; a fix is targeted for a later 9.0 patch, and 9.0.4 is listed as upcoming.³³
- **Redshift cross-Region data lake queries (1 Oct):** provisioned and Serverless Redshift can query S3 data lake tables in other AWS Regions without copying them first.³⁴
- **Glue Data Catalog and Iceberg V3 (1 Oct):** automatic compaction, snapshot and orphan-file clean-up, statistics and crawlers now work for Iceberg V3 tables, including the variant, geospatial and nanosecond-timestamp types.³⁵
- **ElastiCache for Valkey OpenTelemetry metrics (2 Oct):** node-based clusters can publish OpenTelemetry metrics to CloudWatch, free at one-minute resolution or as a detailed 15-second tier, queryable with PromQL.³⁶
- **Snowflake 10.36 (3–7 Oct):** schema-level data metric functions can now skip transient tables, and Native App providers can give users a direct link to an installed app.³⁷

1. [2026-10-07: PostgreSQL JDBC 42.7.14 Security update for multiple CVE's](#), JDBC Project, postgresql.org, posted 9 Oct 2026.
2. [PostgreSQL JDBC Driver](#), pgjdbc project homepage, viewed 10 Oct 2026.
3. [Security](#), pgjdbc project, viewed 10 Oct 2026.
4. [docs: update website for the 42.7.14 release, pgjdbc PR #4473](#), Dave Cramer, GitHub, merged 8 Oct 2026.
5. [CVE-2026-107315](#), CVE record mirror (Strix), published 8 Oct 2026; also [cve.threatint.com](#) for credits.
6. [CVE-2026-107314](#), CVE record mirror (Strix), published 7 Oct 2026.
7. [CVE-2026-107314](#), CVE record mirror (ThreatInt), updated 8 Oct 2026.
8. [CVE-2026-107315: pgjdbc pads a value shorter than its declared length](#), HOL Guard, quoting the upstream advisory.
9. [pgjdbc 42.7.4-42.7.13 Data Padding Flaw & Fix](#), cve.imfht.com.
10. [deps: bump PostgreSQL JDBC driver to 42.7.14 \(stable/8.6\), camunda PR #65329](#), GitHub, merged 9 Oct 2026.
11. [Message Formats](#), PostgreSQL documentation (current).
12. [Interface ByteStreamWriter](#), pgjdbc public API Javadoc.
13. [PostgreSQL JDBC Driver 42.7.4 Released \(changelog\)](#), pgjdbc docs mirrored by Crunchy Data.
14. [PostgreSQL JDBC 42.7.4 Released](#), postgresql.org, 23 Aug 2024.
15. [Postgres 16: require_auth for libpq](#), Michael Paquier, 13 Apr 2023.
16. [PostgreSQL JDBC 42.7.7 Security update for CVE-2025-49146](#), JDBC Project, postgresql.org, 13 Jun 2025.
17. [CVE-2026-54291](#), SUSE security, viewed 10 Oct 2026.
18. [Version Properties](#), Spring Boot 4.0 reference documentation.
19. [CHANGELOG.md](#), pgjdbc, GitHub, viewed 10 Oct 2026.
20. [Announcing Cloudflare K2: serverless event streams](#), Micah Wylde and Marc Selwan, Cloudflare, 1 Oct 2026.
21. [Cloudflare K2 Builds Event Streams on R2 Object Storage, at a Second of Produce Latency](#), Steef-Jan Wiggers, InfoQ, 9 Oct 2026.
22. [Cloudflare K2 overview](#), Cloudflare documentation, viewed 10 Oct 2026.

23. [K2 limits](#), Cloudflare documentation, viewed 10 Oct 2026.
24. [Consume records](#), Cloudflare K2 documentation, viewed 10 Oct 2026.
25. [Durability](#), Cloudflare R2 documentation.
26. [R2 pricing](#), Cloudflare documentation, viewed 10 Oct 2026.
27. [KIP-1150: Diskless Topics](#), Apache Kafka wiki, last updated 2 Mar 2026.
28. [KIP-1150 Accepted, and the Road Ahead](#), Aiven, 12 Mar 2026.
29. [Confluent Acquires WarpStream to Advance Next Gen BYOC Data Streaming](#), Confluent via Business Wire, 9 Sep 2024.
30. [Apache Kafka 4.2.0 Release Announcement](#), Apache Kafka, 17 Feb 2026.
31. [PostgreSQL News Archive](#), postgresql.org, viewed 10 Oct 2026.
32. [Apache Kafka Blog](#), viewed 10 Oct 2026.
33. [Release Notes for MongoDB 9.0: patch releases and known issues](#), MongoDB documentation, viewed 10 Oct 2026.
34. [Amazon Redshift now supports cross-Region queries for your data lake](#), AWS What's New (via aws-news.com), 1 Oct 2026.
35. [AWS Glue Data Catalog now supports table optimization, statistics, and crawlers for Apache Iceberg V3](#), AWS What's New (via aws-news.com), 1 Oct 2026.
36. [Amazon ElastiCache for Valkey now supports OpenTelemetry metrics and detailed monitoring](#), AWS What's New (via aws-news.com), 2 Oct 2026.
37. [Weekly releases 2026](#), Snowflake documentation, 10.36 entry.